

Alex Saccone (acs376)
Tyler White (tjw238)
Alexander Schatzberg (afs223)
Wallace Wei (wzw4)

3110 Final Project, MS3 Report
Boid Meets World
May 16, 2025

Overview

“Boid Meets World” is an ecosystem simulation that demonstrates how disparate predators target a flock of creatures. The prey creatures are simulated with an algorithm inspired by Boids, a program developed by Craig Reynolds in 1986, while the predator creatures follow a set of rules we create ourselves. The prey reproduce when they encounter sufficient food and flee from predators, while the predators are attracted to and consume the boids.

File Structure

Our project is fairly compartmentalized. Here are the units, along with a summary of the functionality they implement.

Main

- This is the entry point to the entire application. It sets up a simulation with the default parameters and launches the GUI, and is also responsible for handling user input in the form of sliders and spawning obstacles/predators from mouse inputs.

SimParams

- Each function that updates the simulation state accepts a list of parameters (e.g., max speed, max vision, mouse attraction, etc) that dictate how the simulation will run, and this is where these parameters are stored. The user can modify these parameters with sliders in the GUI. This module defines those parameters and methods for getting and updating them.

GUI

- This module is responsible for drawing the screen, including all its components like food, boids, predators, and obstacles. This was implemented using the OCaml bindings for Raylib and Raygui.¹

Boids

- Responsible for the logic of the interactions between boids and their movements; this is also the compilation unit that uses the effects of the obstacles and mouse cursor, as it's where boid velocities are calculated each time step

Food

¹ <https://github.com/tjammer/raylib-ocaml>

- Responsible for managing a single piece of food (including its position and amount) and the set of all food on the screen.

Hungry

- Responsible for adding the additional functionality that boids eat food. Boids have a food status, and they eat food if close enough during the simulation.

Obstacle

- Responsible for handling the obstacles that can be spawned with a left-click and the mouse cursor repulsion/attraction

Predator

- A boid-like object whose goal is to kill boids in various random methods, which the boids try to avoid

Reproduce

- Responsible for enabling boids to reproduce if they have enough food stored up. The new baby boid spawns close to them.

Extras

- Several useful general functions that could be used in several compilation units, such as printing a list according to a printer function, removing an element at a certain index from a list, and performing tuple arithmetic.

Testing

Here is our strategy for each module:

Main/GUI

- No unit testing was done here; the program was simply run to ensure it looked and operated as intended.

SimParams

- Some unit testing was performed to ensure that getting and updating each parameter worked as intended, including getting the default for each parameter when parameters are reset. Additionally, tests were run visually to determine suitable ranges and default values for each parameter.

Boids

- Many specification tests are conducted for individual boids and flocks. These include making boids and making sure they match the requirements and adding and removing boids from flocks, and checking the effects.

Food

- Most tests were quite basic (creating, setting, adding, and removing food, etc.) since food only has one involved function. There were some list-like specification tests to ensure that adding and removing food worked as required. Additionally, there was a

specification test for adding food to a foodset and comparing the highest food amount of the previous to the new one.

Hungry

- Testing was somewhat similar to boids since hungry boids are enhanced boids. However, there were also tests ensuring that the simulation with hungry boids functioned just like the simulation with boids when the food aspect was ignored.

Obstacle

- Testing primarily involved spawning a number of obstacles in an environment, calculating what their expected effect would be on a given boid, and ensuring that it matched up with the effect actually demonstrated on the boid.

Predator

- Testing looked somewhat similar to boids, as they do share some properties with boids, but also made sure to test the simulation part in a way to see if the same number of predators remained after a time step, on top of the basic operations present, which were tested.

Reproduce

- The only tests ensured that reproduction with boids worked reasonably well compared to the simulation without reproduction. This is done by comparing the number of boids before and after a step of simulation and making sure that the boid count does not increase by more than 100% (since each boid can reproduce at most one child during a step).

Extras

- Testing was not explicitly done for this file, but it was tested for through the testing of other files.

Progress

Since MS2, we have added mouse attraction (boids flocking towards the mouse), the simulation now has continuous “sliders” for various things like the various parameters of the “boids” like the coefficients for the three rules: alignment (velocity), cohesion (position), and separation (position), as well as other parameters for the newly added features, all available.

On top of this, we have also added predators, food, boid reproduction, and obstacles. The food is a physical object that the boid can “eat”, and once they have eaten enough, they reproduce asexually. To balance this, they will face off against obstacles and predators, with obstacles being able to be placed with the “left click”, which will naturally repel boids from the location based on its adjustable strength and radius of effect, and predators placed with a “right click” that will chase after the boids once a boid is within the predator’s vision radius and kill them.

Note that here, the simulation has little green dots as food, red triangles as the boids, black dots as the obstacles, and the bigger, purple triangles as the predators.

Individual Activity Breakdown

Alex Saccone

- Wrote and tested obstacles
- Wrote and tested cursor interaction
- Processed mouse input
- Integration of the above components into boids, gui, and simparams
- Tweaks to GUI/boids/predators here and there
- ~20 hours spent on project between MS2 and MS3

Tyler White

- Wrote and tested food, hungry boids, and reproduction (including refactoring them)
- Worked on the GUI representation of food
- Wrote most of the extras file
- Small contributions in other places
- Spent about 30 hours on the project from MS2 to MS3

Zander Schatzberg

- Wrote the basic GUI, driver program, and simParams module
- Tested the simParams module
- Handled mouse input
- Spent ~20 hours on the project

Wallace Wei

- Wrote Predators/Tested It, did some of Boids, obviously with some other small contributions to help integrate these to gui and such
- 20 hours spent working on the project

Productivity Analysis

Overall, the work was distributed fairly evenly. While we did complete a working final project, there were a few holes in our process. We've outlined our primary shortcomings below.

Git Usage

- We utilized git more effectively this time around, but there were still some issues with branches persisting for too long. This was problematic with Food and Reproduce, as it led

to a large number of merge conflicts and some extremely redundant code that more group members didn't get eyes on until too late.

Communication

- Communication was fairly decent this time around, but some group members were on different pages with how some features should be implemented. This, in conjunction with overdeveloped branches, led to more rewriting than necessary in the final stages of the project.
- There were definitely some communication issues with how food/hungry/reproduce would be combined with the original implementation of boids, since it was written with the intention to be used as a toggleable feature, but this did not end up being toggleable. This created a lot of merge conflicts.
- There should have been more communication when merging into main (and especially changing function signatures of elements present in MS2) since this created a lot of merge conflicts.

Sprints (or lack thereof)

- We all got fairly busy toward the end of the semester (shocker), and our sprint structure deteriorated. We procrastinated quite a bit of work, which left more to do than desirable during finals week.

Potential Expansion

If we continue the project, these are the things that we might add.

Conceptual

- Predator reproduction/starvation to mirror a more realistic ecosystem
- Boid attraction to food (too computationally expensive with our naive implementation; also ties into future performance improvements)
- Different species of boids that interact with other species differently from their own

Performance

- Multithreading to improve simulation speed
 - May involve tiling the simulation into sectors to increase the number of boids the simulation could handle concurrently